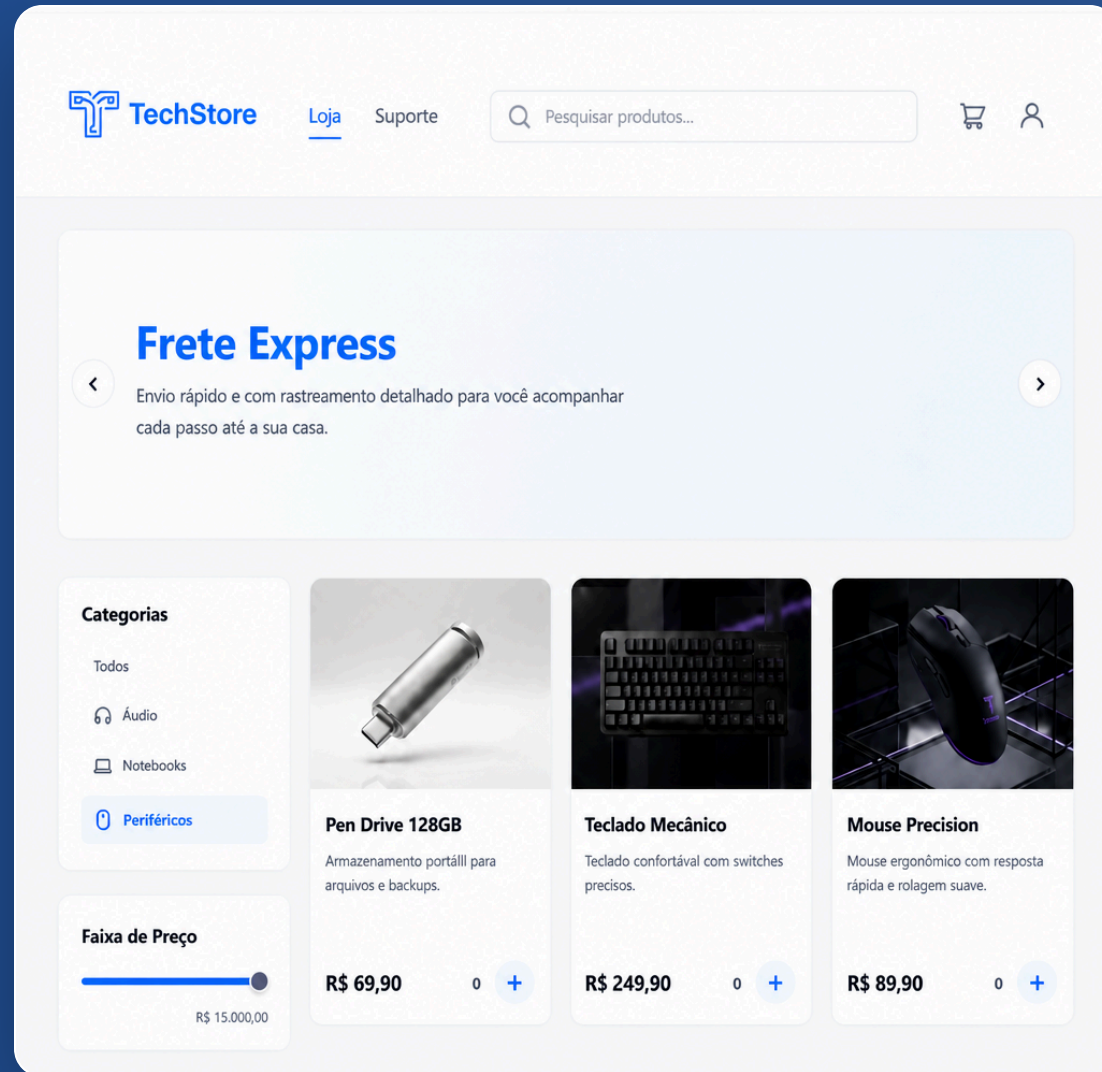


TechStore

Persistência e Gestão de Dados

"Uma loja demo onde os dados sobrevivem ao ciclo de vida efêmero da infraestrutura."



O problema e quem é impactado

Banco de dados acoplado ao contêiner da aplicação: cada reinício vira perda de dados.

SEM PERSISTÊNCIA · CENÁRIO DE RISCO

Acoplamento efêmero

O banco sobe e desce junto com o contêiner da aplicação.

Perda total de dados

Reiniciar, atualizar ou recriar o contêiner apaga tabelas, sessões e produtos.

Impacto operacional

Inviabiliza a operação, com perda irreparável a cada manutenção.

QUEM É IMPACTADO

C

Cliente

Perde o carrinho e a sessão a cada reinício, e repete o cadastro.

A

Administração

Perde o histórico de pedidos e não consegue auditar vendas.

O

Operação

Cada manutenção vira risco de negócio, sem garantia de retorno.

A solução proposta

Desacoplar a camada de dados do ciclo de vida da aplicação e integrá-la ao processo de entrega.

CAMADA DE DADOS ISOLADA

- **PostgreSQL 15** em contêiner dedicado e independente da aplicação.
- Volume nomeado `pg_data` montado fora do contêiner, com ciclo de vida próprio.
- `docker compose down` sem `-v` remove a aplicação e **preserva 100% dos dados**.
- Rede dedicada `techstore_net`, sem porta do banco exposta ao host.

ENTREGA E OPERAÇÃO

- **Bootstrap automático:** `entrypoint.sh` aplica migrações e popula o banco na subida.
- **Healthcheck e readiness:** a API só recebe tráfego com o banco saudável.
- **Integridade no banco:** 5 constraints `CHECK` e 4 índices.
- **Recuperação de senha:** token gravado apenas como hash, com expiração de 1 hora.

Como o MVP funciona

Um comando sobe a stack inteira. O volume sobrevive ao ciclo de vida dos contêineres.

Comando único

```
docker compose up -d --build
```

Bootstrap

```
entrypoint.sh
```

 aplica migrações e popula o banco.

Healthcheck

A API só recebe tráfego após o banco ficar *healthy*.

Acesso

```
localhost:3001
```

 interface ·

```
localhost:3002
```

 API

Escalabilidade

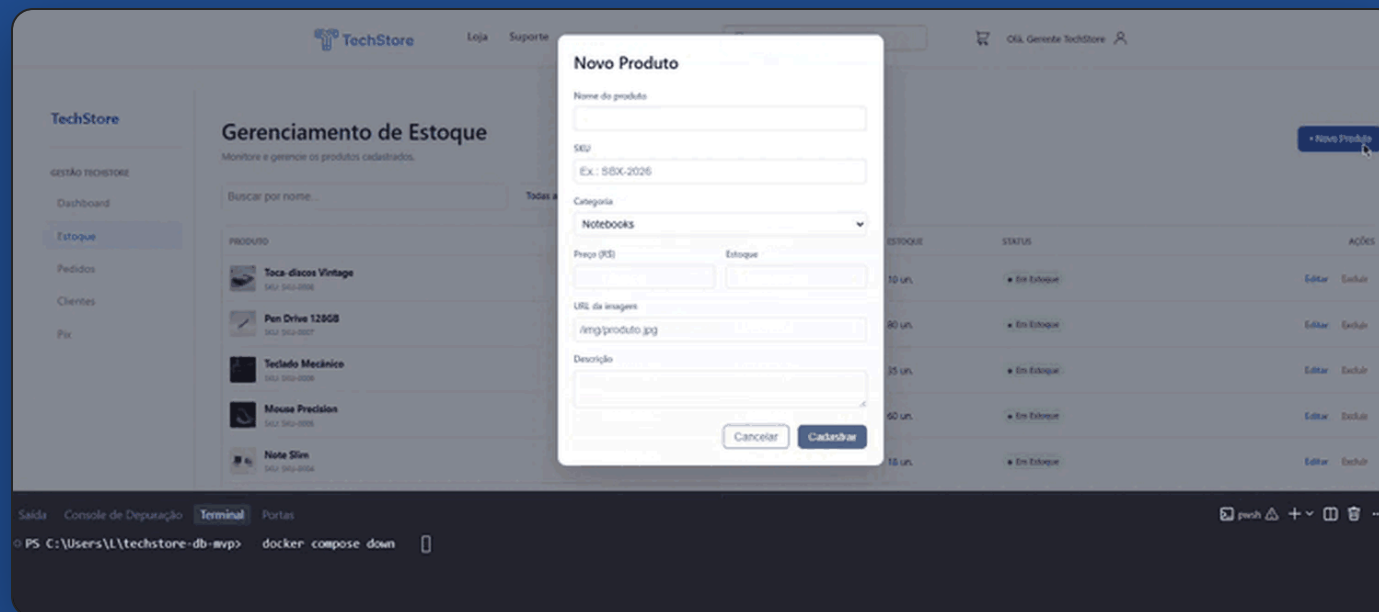
Checkout transacional com controle de concorrência e idempotência.

CICLO DE VIDA INDEPENDENTE

docker compose down volume preservado

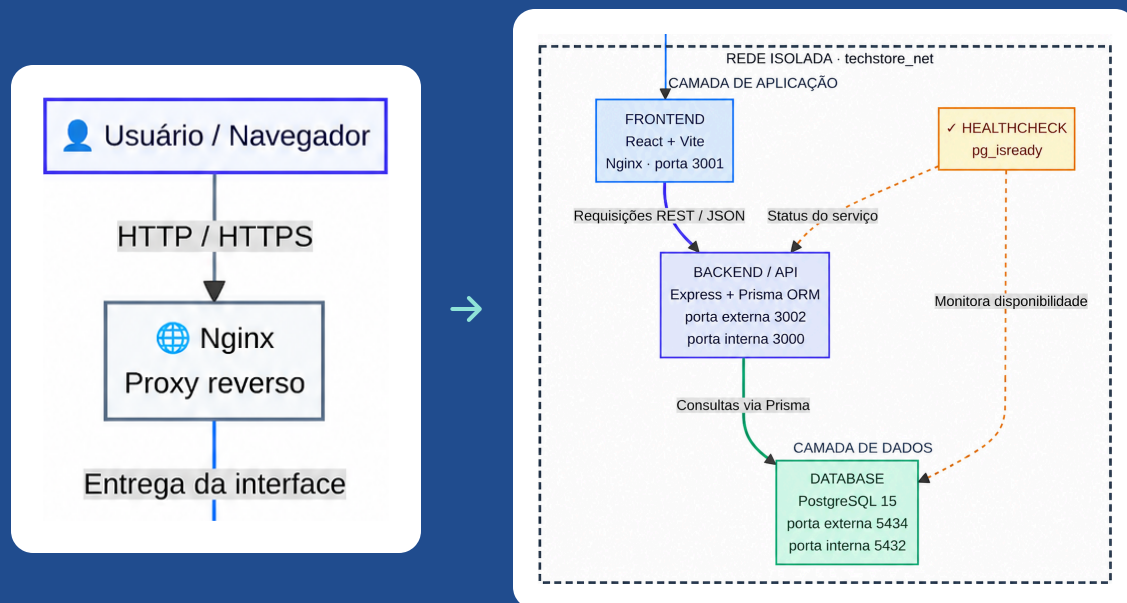
docker compose up -d volume reutilizado

8 produtos e 2 usuários, sem duplicar registros.



Os dados permanecem no PostgreSQL depois de derrubar e recriar a stack.

Como foi construído, com o quê e por quê



PRÁTICAS E FERRAMENTAS DEVOPS

- **Docker e Docker Compose**
stack como código

- **Healthcheck e readiness**
sem corrida na subida

- **Bootstrap automatizado**
seed idempotente

- **Volume nomeado**
dados fora do contêiner

- **CI/CD · 3 workflows**
ci.yml test-docker-stack.yml protecao-main.yml

- **Backup e restauração**
scripts de verificação

DECISÕES TÉCNICAS E POR QUÊ

- **PostgreSQL 15**
ACID e integridade

- **Express com Prisma**
consultas type-safe

- **Nginx como proxy**
estáticos e API

- **Rede e portas restritas**
banco isolado

- **Checkout transacional**
sem oversell

Resultados obtidos e próximos passos

RESULTADOS COM O MVP

- 01 Arranque zero-config**
Stack inteira em um comando Docker Compose.
- 02 Bootstrap idempotente**
8 produtos e 2 usuários sem duplicar registros.
- 03 Persistência e integridade verificadas**
Dados sobrevivem à recriação da stack; o checkout impede oversell.
- 04 Qualidade comprovada**
10 suítes e 35 testes, 6 tabelas e 16 endpoints REST.

PRÓXIMOS PASSOS E EVOLUÇÃO

- 01** Reserva e expiração de estoque
- 02** Histórico de pedidos e pagamentos
- 03** Ledger de estoque para auditoria
- 04** Migrações sem indisponibilidade

DECISÃO TÉCNICA EM DESTAQUE

O token de recuperação de senha é armazenado apenas como **hash**, com expiração de 1 hora.

OBRIGADO PELA ATENÇÃO

Obrigado!

O projeto prova que a perda de dados não é um problema inevitável de infraestrutura, e sim uma decisão de arquitetura.

LG

Lays Gomes

Apresentação

FA

Fellype Augusto

Apresentação

IT

Ivan Teotônio

Apresentação

VK

Victor Kazu

Apresentação

TechStore · MVP de Persistência e Gestão de Dados · **DevOps** · **Aponti** · 05/10/26

github.com/laas26/techstore-db-mvp